

OPERATORS AND EXPRESSION

BY SUJAL SHRESTHA

C OPERATORS

An operator is simply a symbol that is used to perform Different operations. There can be many types of operations like arithmetic, logical, bitwise, etc. There are following types of operators to perform different types of operations in C language.

- Arithmetic Operator
- Relational Operator
- Logical Operators
- Bitwise Operators
- Assignment Operator
- Ternary or Conditional Operators
- Increment/Decrement Operator
- Sizeof Operator

ARITHMETIC OPERATORS

An arithmetic operator performs mathematical operations such as addition, subtraction, multiplication, division etc on numerical values (constants and variables).

- + Addition
- - Subtraction
- * Multiplication
- / Division
- % Modulo Division

RELATIONAL OPERATORS

A relational operator checks the relationship between two operands. If the relation is true, it returns 1; if the relation is false, it returns value 0. Relational operators are used in decision making and loops.

- > Greater than
- < Less than
- >= Greater than equal to
- <= Less than equal to
- == Double equal to
- != Not equal to

LOGICAL OPERATOR

An expression containing logical operator returns either 0 or 1 depending upon whether expression results true or false. Logical operators are commonly used in decision making in C programming.

- && (logical AND)
- || (logical OR)
- ! (logical NOT)

BITWISE OPERATORS

Bitwise operators are used in C programming to perform bit-level operations. During computation, mathematical operations like: addition, subtraction, multiplication, division, etc are converted to bit-level which makes processing faster and saves power.

- $\&$ (Bitwise AND)
- $|$ (Bitwise OR)
- $\wedge$ (Bitwise X-OR)
- $\sim$ (Bitwise complement)
- $>>$ (Bitwise Right Shift)
- $<<$ (Bitwise Left Shift)

BITWISE OPERATORS

- Bitwise $\&$, $|$, $\wedge$, $>>$, $<<$ are used for two operand data. But $\sim$ is only used for single operand data.
- The result of the bitwise AND operation is 1 if both the bits have the value as 1; otherwise, the result is always 0.
- The result of the bitwise OR operation is 1 if at least one of the expression has the value as 1; otherwise, the result is always 0.
- The result of the bitwise Exclusive-OR operation is 1 if only one of the expression has the value as 1; otherwise, the result is always 0.

BITWISE OPERATORS

x	y	$x \& y$	$x y$	$x \wedge y$
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

BITWISE OPERATORS

- The bitwise complement Operator ($\sim$) is also called as one's complement operator since it always takes only one value or an operand. It is a unary operator. When we perform complement on any bits, all the 1's become 0's and vice versa.

Eg:

- int a=10, $\sim$ a=-11

int a=-10, $\sim$ a=9

BITWISE OPERATORS

- The left shift operator (<<) is a type of Bitwise shift operator, which performs operations on the binary bits. It is a binary operator that requires two operands to shift or move the position of the bits to the left side and add zeroes to the empty space created at the right side after shifting the bits.

Syntax:

variable_name << no_of_position

Example

int a=20, a<<2 80

BITWISE OPERATORS

- The right shift operator ($\gg$) is a type of bitwise shift operator used to move the bits at the right side, and it is represented as the double ($\gg$) arrow symbol. Like the Left shift operator, the Right shift operator also requires two operands to shift the bits at the right side and then insert the zeroes at the empty space created at the left side after shifting the bits.

Syntax:

`variable_name >> no_of_position`

Example

`int a=20, a>>2 = 5`

OPERATORS

Assignment Operator

An assignment operator is used for assigning a value to a variable. The most common assignment operator is =.

- =
- +=
- -=
- *=
- /=
- %=

OPERATOR

Conditional/Ternary Operator

The operators, which require three operands to act upon, are known as ternary operators. It can be represented by "`? :`". It is also known as conditional operator. The operator improves the performance and reduces the line of code.

Syntax

`Expression1 ? Expression2 : Expression3`

Exmple

`a < b ? a : b`

OPERATORS

Increment and Decrement Operators

C programming has two operators increment `++` and decrement `--` to change the value of an operand (constant or variable) by 1.

Increment `++` increases the value by 1 whereas decrement `--` decreases the value by 1. These two operators are unary operators, meaning they only operate on a single operand.

OPERATORS

```
1  #include <stdio.h>
2  int main() {
3      // Write C code here
4      int a=5, b=5, c=5,d=5;
5      printf("\n Pre Increment %d ", ++a); // a becomes 6, then prints 6
6      printf("\n Post Increment %d ",b++); // prints 5, then b becomes 6
7      printf("\n Pre Decrement %d ", --c); // c becomes 4, then prints 4
8      printf("\n Post Decrement %d ",d--); // prints 5, then b becomes 4
9      printf("\n %d %d ", a, b);
10     printf("\n %d %d", c, d);
11     return 0;
12 }
```

Pre Increment 6
Post Increment 5
Pre Decrement 4
Post Decrement 5
6 6
4 4

=== Code Execution

OPERATORS

sizeof() operator in C

The sizeof() operator is commonly used in C. It determines the size of the expression or the data type specified in the number of char-sized storage units. The sizeof() operator contains a single operand which can be either an expression or a data typecast where the cast is data type enclosed within parenthesis.

The data type cannot only be primitive data types such as integer or floating data types, but it can also be pointer data types and compound data types such as unions and structs.

Example:

```
int a=20;
```

```
sizeof(a) or sizeof(int)
```


EXPRESSION

An expression is a statement in which operands are linked to each other by the use of operators to compute a value. An operand can be a function reference, a variable, an array element or a constant.

Example:

$C = a + b;$

There are four types of expression in C Programming

- Arithmetic expressions
- Relational expressions
- Logical expressions
- Conditional expressions

$x = 9/2 + a - b, 6 * 2 / (2 + 1 * 2/3 + 6) + 8 * (8/4)$

PRECEDENCE OF OPERATORS IN C

The precedence of operator specifies that which operator will be evaluated first and next. The associativity specifies the operator direction to be evaluated; it may be left to right or right to left.

Let's understand the precedence by the example given below

```
int a=10+20*10;
```

The a variable will contain 210 because * (multiplicative operator) is evaluated before + (additive operator).

PRECEDENCE OF OPERATORS IN C

The precedence and associativity of C operators is given below:

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Left to right
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Left to right
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Left to right
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right

TYPECASTING IN C

Typecasting allows us to convert one data type into other. In C, when you convert the data type of a variable to another data type then this technique is known as typecasting.

Types of typecasting

- Implicit Typecasting
- Explicit Typecasting

TYPECASTING IN C

Implicit Typecasting

Implicit Typecasting means data is converted into another data type automatically by the compiler. Programmer doesn't need to do anything explicitly.

Rule for Implicit typecasting

- If you are doing a conversion on two different data types then the lower data type is automatically converted into a higher data type.
- Suppose, you are doing type casting operation between two data types such as int and float then the value of the result would be floating type

TYPECASTING IN C

Implicit Typecasting

```
#include <stdio.h>
```

```
int main() {
```

```
    int a = 4, x;
```

```
    float b = 12.4, c;
```

```
    c = a + b; // Implicit conversion from int to float
```

```
    x = a + 'd'; // Implicit conversion from char to int
```

```
    printf("%f\n", c); // Prints the float value
```

```
    printf("%d\n", x); // Prints the integer value
```

```
    return 0;
```

```
}
```

EXPLICIT TYPECASTING

It is not like implicit type casting where the data type is converted automatically. In Explicit type casting, you have to force the conversion between data types.

Syntax:

`(data_type) expression;`

Where,

- `data_type` is the name of the data type to which you want to convert to.
- The expression can be a variable, an actual expression or a constant.

EXPLICIT TYPECASTING

```
#include <stdio.h>
int main() {
float val = 56.3;
int a = (int)val + 50; // explicit type casting
printf("Value of a: %d\n", a);
return 0;
}
```


INPUT AND OUTPUT

Output:

To take output in C language, printf() function is used:

Eg.

```
printf("Hello World");
```

In order to output variable value, the format specifier is used for that variable. Format Specifiers for integer, character and float data type are %d, %c and %f respectively.

Eg.

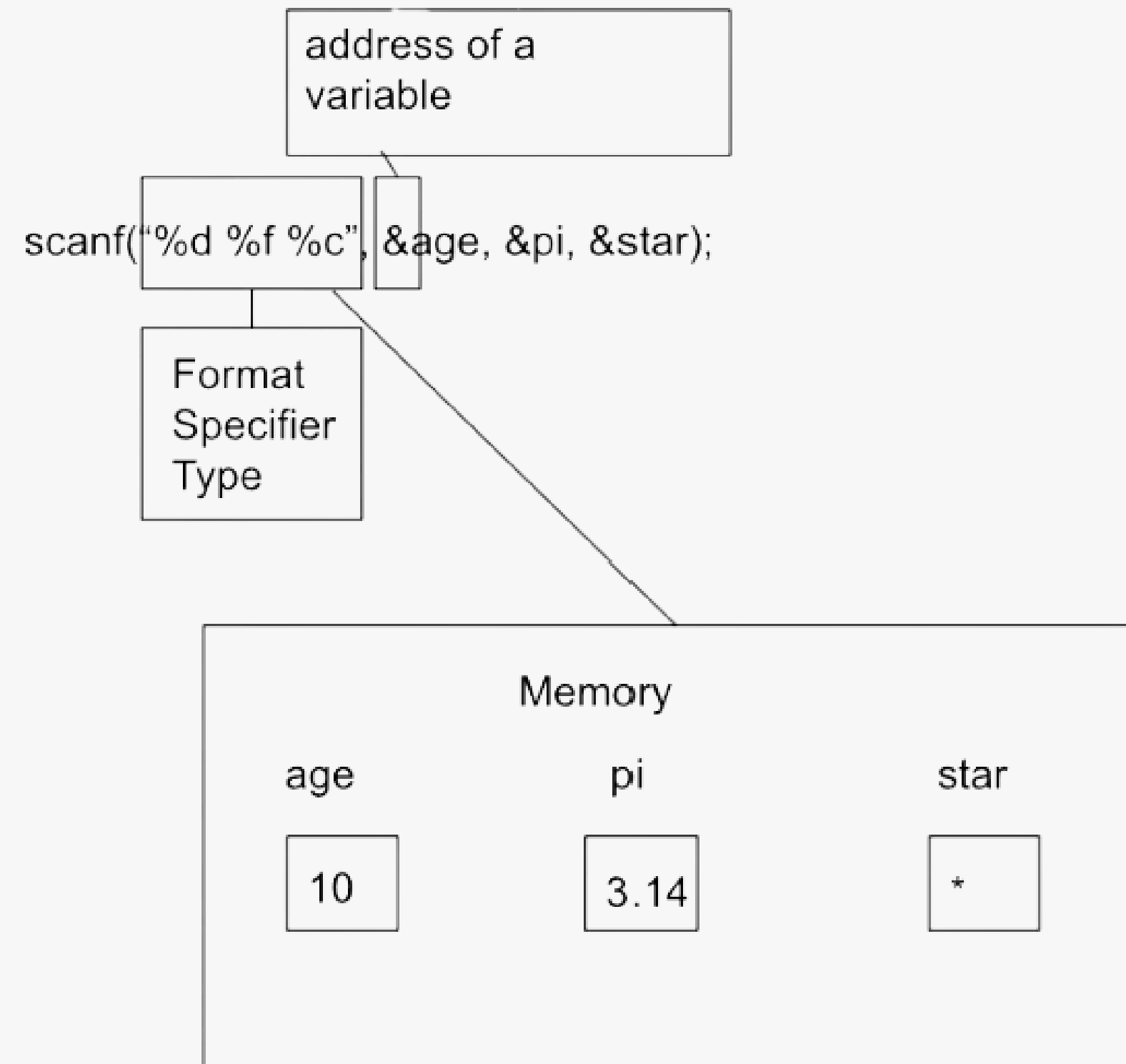
```
int price = 2;
```

```
char star = '*';
```

```
float pi=10.52;
```

```
printf("The price of %d %c is %f", price, star, pi);
```

INPUT AND OUTPUT



THANK YOU

